



**Department of Electrical,
Computer, & Biomedical Engineering**
Faculty of Engineering & Architectural Science

Course Title:	Digital Systems
Course Number:	COE328
Semester/Year (e.g.F2016)	F2022

Instructor:	Dr. Reza Sedaghat
--------------------	-------------------

<i>Assignment/Lab Number:</i>	6
<i>Assignment/Lab Title:</i>	Design of a Simple Central Processing Unit

<i>Submission Date :</i>	Thursday, December 12, 2024
<i>Due Date:</i>	Thursday, December 12, 2024

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Arizaga	Andrew	501225485	11	A.A.
Malik	Haziq	501260350	11	Haziq Malik

Section 1: Table of Contents

1. Table of Contents
 - All chapters/sections with page numbers
2. Introduction (Page 3)
 - Overview of the components
3. Components(Page 3)
 - Description
 - Truth Table
 - Circuit Diagram/Block Diagram
 - Waveform File
4. ALU_1 (Problem Set 1)(Page 9)
 - Purpose and Design
 - Block Schematic File (BDF)
 - Inputs and Outputs
 - Microcode Table
 - Waveform File
5. ALU_2 (Problem Set 2)(Page 10)
 - Purpose and Design
 - Block Schematic File (BDF)
 - Inputs and Outputs
 - Microcode Table
 - Waveform File
6. ALU_3 (Problem Set 3)(Page 12)
 - Purpose and Design
 - Block Schematic File (BDF)
 - Inputs and Outputs
 - Microcode Table
 - Waveform File
7. Conclusion(Page 13)
 - Summary and Reflection

Section 2: Introduction

This laboratory was about making a simplified central processing unit using VHDL in the Quartus software development environment and implementing the CPU on an fpga board. This laboratory combined several smaller components from previous laboratories. These components consisted of storage elements like the registers which stored the 8-bit numbers that were to be operated on. The combinational logic elements which included the ALU that performed arithmetic operations on stored numbers. The controlling elements such as the FSM and the decoder, whose purpose was to select a function. Then the display elements like the seven segment display which outputs the numbers visually. Overall, the objective of this laboratory was to combine these components in order to make a simplified CPU.

Section 3: Components

Register:

This is a storage element that stores a binary number. In our case it was 8-bits and it changes the value based on a clock trigger. In this lab there were 8 D type flip flops which were used in each register.

Clk	D	Q(t+1)
0	x	Q(t)
1	0	0
1	1	1

Table 1: Truth table for the two registers

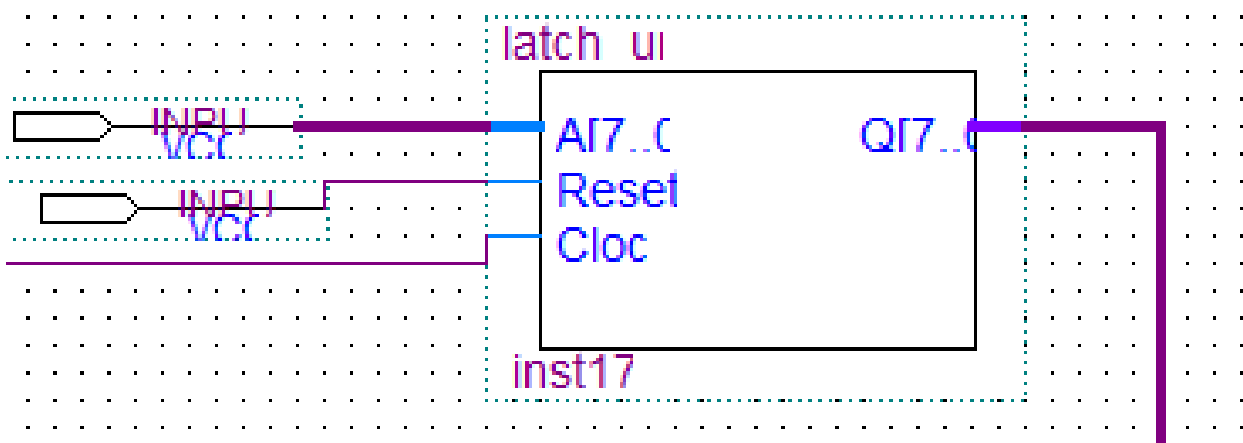


Figure 1: Block diagram of register

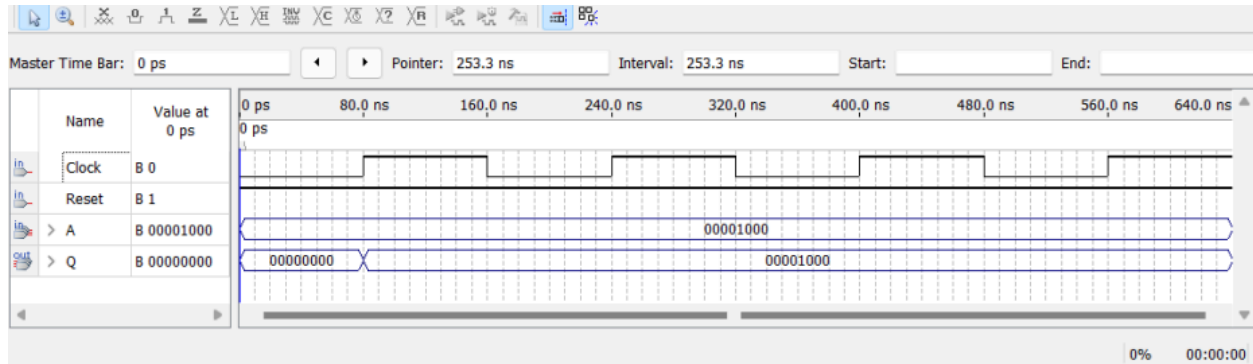


Figure 2: Waveform simulation of register

Finite State Machine:

The finite state machine in this laboratory was the Moore machine which goes through the states S_0 to S_8 by using a clock. Which means when the data in is a 1 the FSM goes from the current state to the next state and if the data in value 0 it stays at the current state. Each state is also mapped to a digit of the student ID so when a state is set a student digit is output in the respective order. In total there are 9 states which are cycled through and each time when S_8 is reached the Moore machine returns back to S_0 . Also the states are used as inputs for the decoder which gives microcode to the ALU. In summary, the FSM basically acts as a controlling sequencer for the ALU.

Present State	Next State		Output: Student ID
	Data IN=0	Data IN=1	
0000	0000	0001	0101
0001	0001	0010	0000
0010	0010	0011	0001
0011	0011	0100	0010
0100	0100	0101	0010
0101	0101	0110	0101
0110	0110	0111	0100
0111	0111	1000	1000
1000	1000	0000	0101

Table 2: Truth table for the Finite State Moore Machine

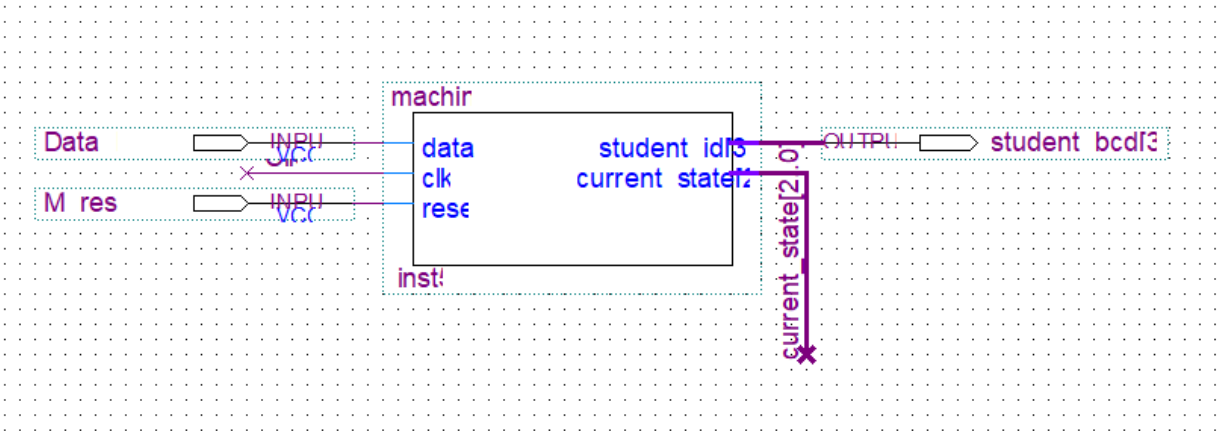


Figure 3: Block Diagram of FSM

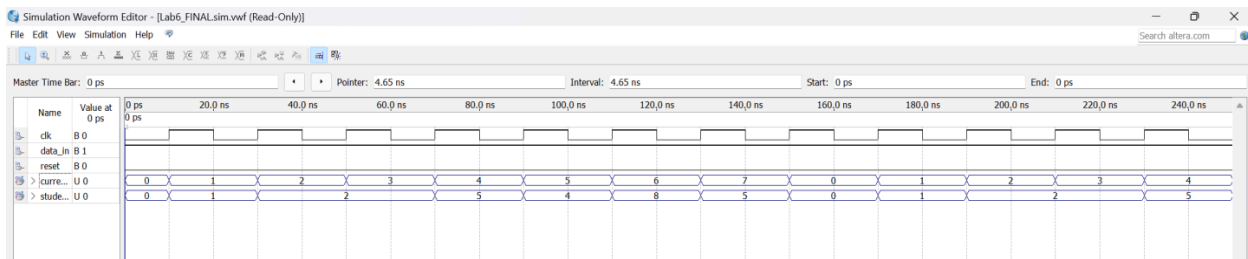


Figure 4: Waveform simulation of the FSM

Decoder:

The decoder takes a 4 bit input which is the state of the FSM and instead outputs a 16 bit number which is the input for the OP code for the ALU. In summary, this component basically acts like a translator between the FSM output and the input for the ALU opcode.

X ₀	X ₁	X ₂	X ₃	Y ₀	Y ₁	Y ₂	Y ₃	Y ₄	Y ₅	Y ₆	Y ₇	Y ₈	Y ₉	Y ₁₀	Y ₁₁	Y ₁₂	Y ₁₃	Y ₁₄	Y ₁₅
0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
0	1	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
0	1	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
0	1	1	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0

1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Table 3: Truth table for a 4-16 Decoder

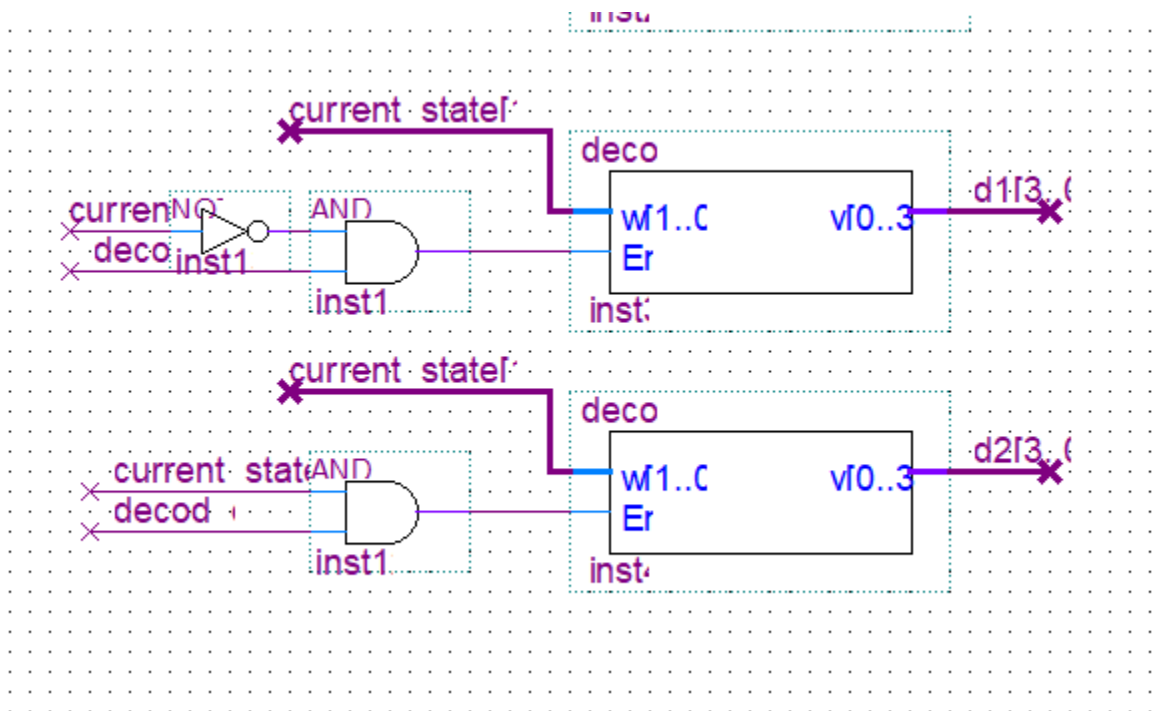


Figure 5: Block Diagram for Decoder

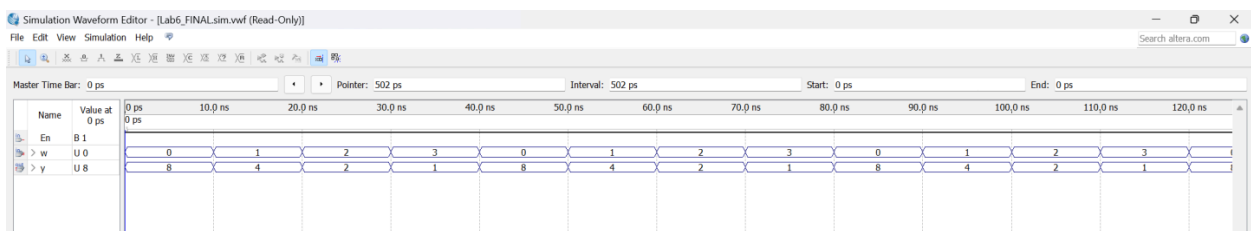


Figure 6: Waveform simulation of the Decoder

ALU:

The arithmetic logic unit performs operation based on the OPcode that is imputed in it for which the opcode is dictated from the decoder which itself gets its values from the FSM. The ALU for problem 1 has 8 operations which were taken from the lab manual which included addition subtraction, and other logical related operations. The ALU has the inputs Clock, A, B, and Opcode. Also when the clock changes from 0 to a 1 in a rising edge manner the ALU checks the Opcode then the corresponding function is executed which does an operation on A and B.

The ALU then outputs Neg, R1, R2 where R1 and R2 are each 4 bits for the two seven segment displays. Additionally, there were two more modifications to the ALU for problem 2 and 3, where for problem 2 the ALU the logical operations/function were changed such that function b was implemented, while for problem 3 the ALU was fed the student ID outputting 'y' if the signal was even and 'n' if that was not the case. In summary, the ALU does the actual operations in the CPU.

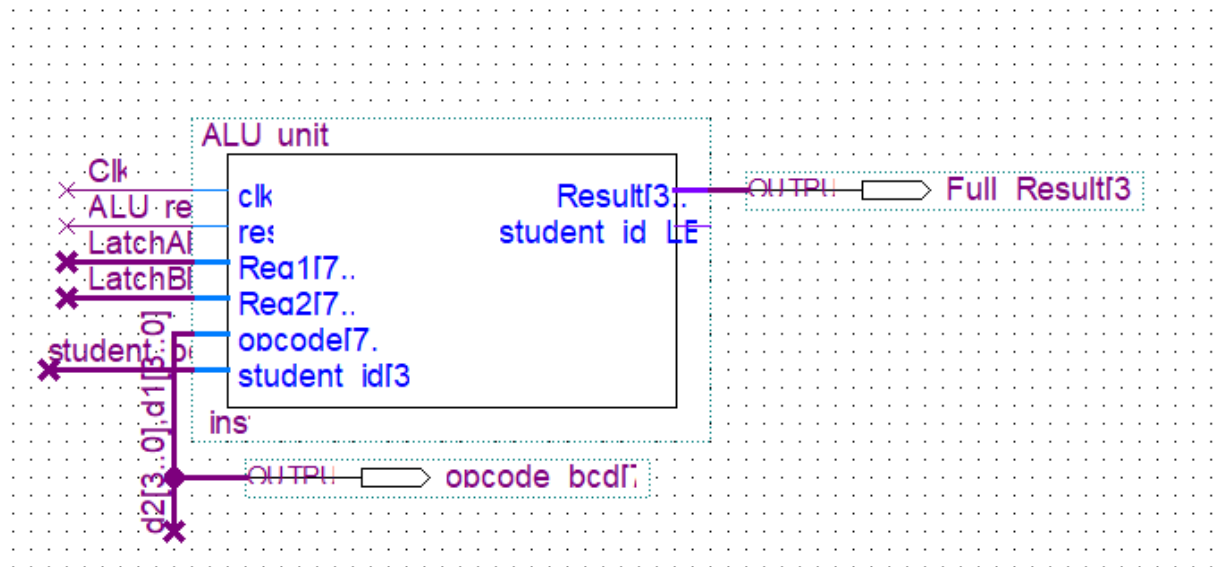


Figure 7: Block Diagram for the ALU

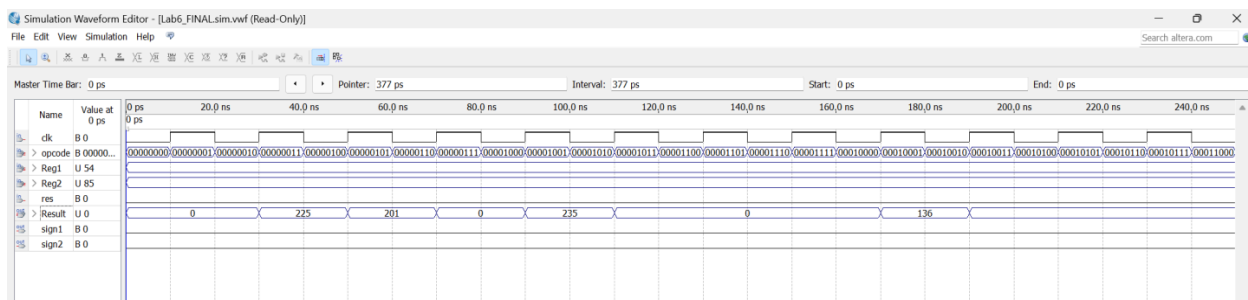


Figure 8: Waveform simulation of the ALU

Seven Segment Display:

The seven segment display or sseg for short display values from 0-F in hexadecimal. In this case of this lab 2 seven segment displays were used for the output from the ALU in order to display the result after an operation was performed on the number A and B.

bcd(input)	leds(abcdefg)
0000	1111110
0001	0110000
0010	1101101
0011	1111001
0100	0110011
0101	1011011
0110	1011111
0111	1110000
1000	1111111
1001	1111011
1010	1110111
1011	0011111
1100	1001110
1101	0111101
1110	1001111
1111	1000111

Table 4: Truth table for a seven segment display

neg	leds(abcdefg)
0	0000000
1	0000001

Table 5: Truth table for a seven segment display with negative input

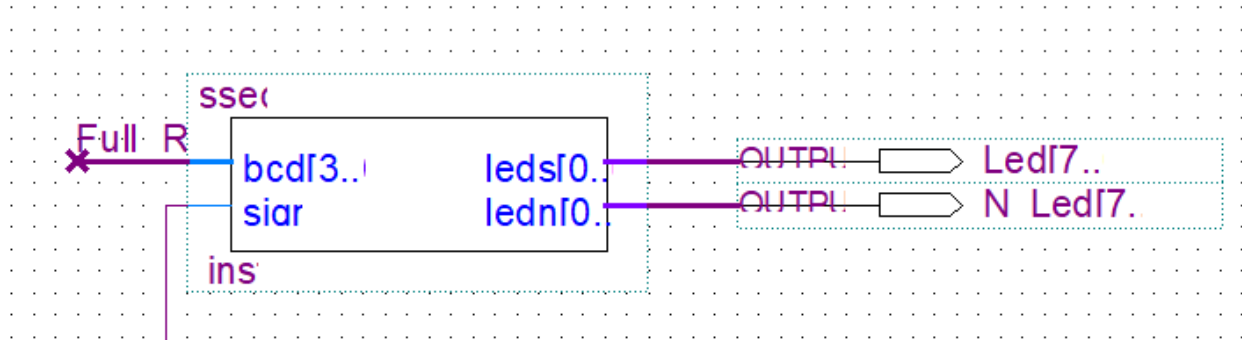


Figure 9: Block diagram for Seven Segment Display

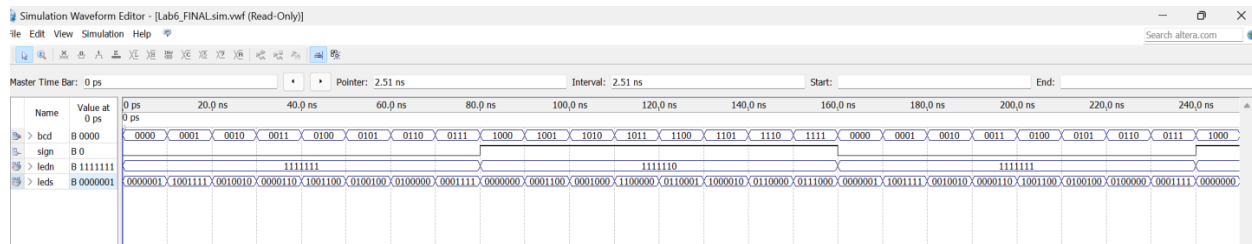


Figure 10: Waveform simulation of the Seven Segment Display

Section 4: ALU_1 Problem 1

For this problem of the laboratory all that needed to be done was implement the ALU functions and the Student ID was not necessary as an input into the ALU. For this specific problem the logical functions that were performed. For this ALU there are for the inputs, the clock signal, the reset signal, the register 1 & register2 8-bit inputs, and an Opcode/microcode input. The output was R1 and R2 which were each 4 bits for a combined total of 8-bits, and an output to dictate if the result was negative. The clock signal was a rising edge where each logical function was executed when the clock signal was 1.

Function #	Opcode	Function
1	00000001	$sum(A, B)$
2	00000010	$dif(A, B)$
3	00000100	$\overline{A}$
4	00001000	$\overline{A} \cdot \overline{B}$
5	00010000	$\overline{A} + \overline{B}$
6	00100000	$A \cdot B$
7	01000000	$A \oplus B$
8	10000000	$A + B$

Table 6: Opcode table for ALU problem 1

[illegible]

Section 5: ALU 2 Problem 2

Function #	Opcode	Operation/Function
1	00000001	Sway lower and upper 4 bits of A
2	00000010	Produce result of A OR B

3	00000100	Decrement B by 5
4	00001000	Invert all bits of A
5	00010000	Invert bit-significance order of A
6	00100000	Max(A,B)
7	01000000	Produce difference of A-B
8	10000000	Produce result of A XNOR B

Table 7: Function table for ALU problem 2, function table b

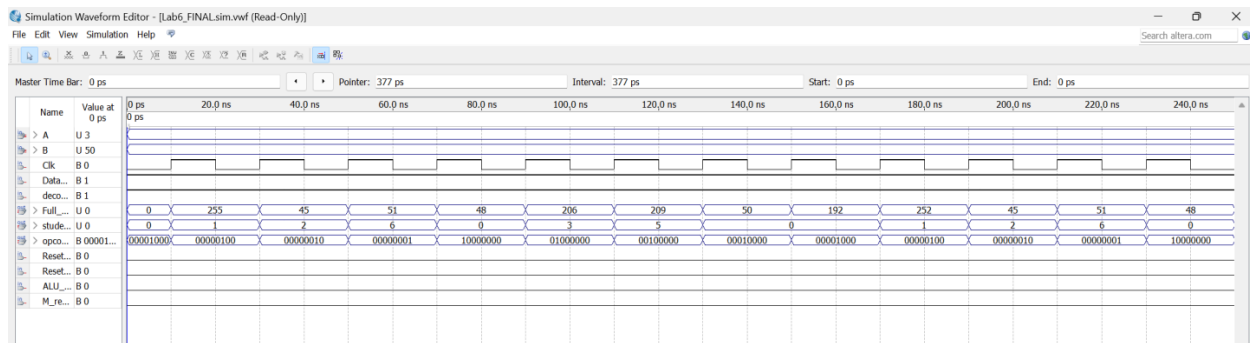


Figure 13: Waveform for ALU problem 2

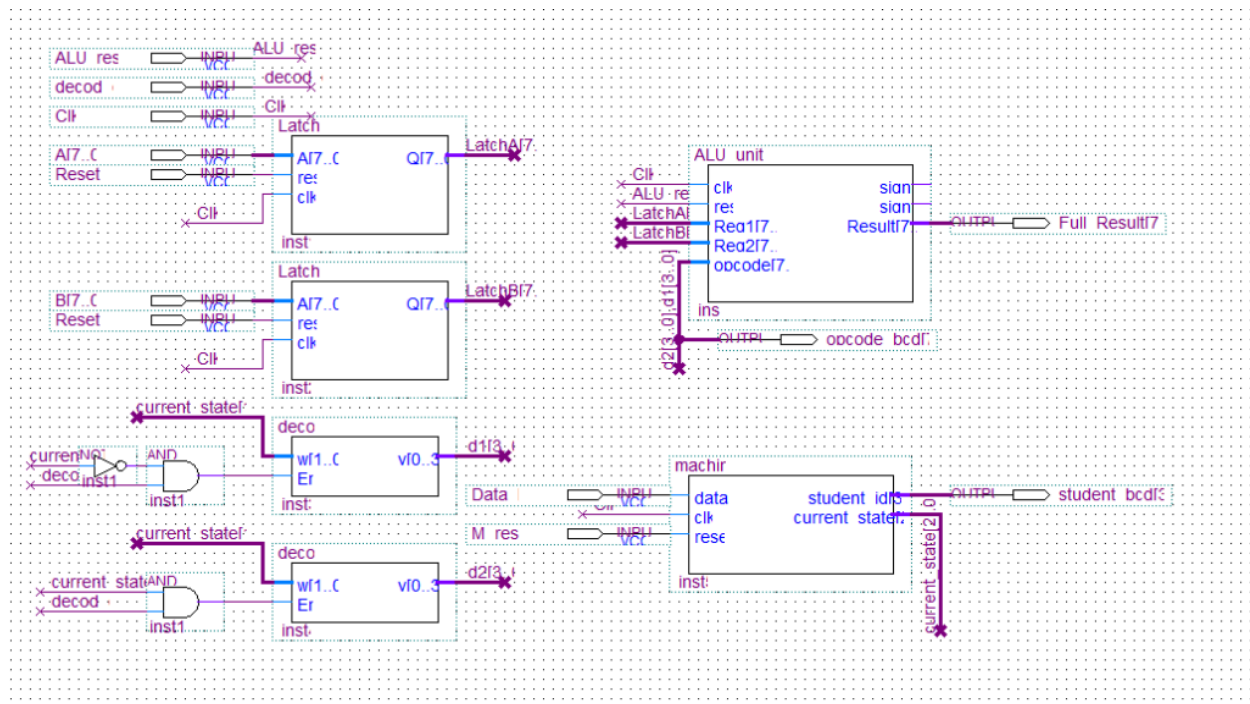


Figure 14: BDF for ALU problem 2

Section 6: ALU_3 Problem 3

For this problem, the logical operation for the ALU was changed such that when the Student ID was input into the ALU it would return 'y' if the signal was even and 'n' if the signal was odd. are for the inputs, the clock signal, the reset signal, the register 1 & register2 8-bit inputs, and an Opcode/microcode input, but most importantly also the StudentID. The output was R1 for yes or no which was fed into the modified seven segment display which outputted 'y' or 'n'.

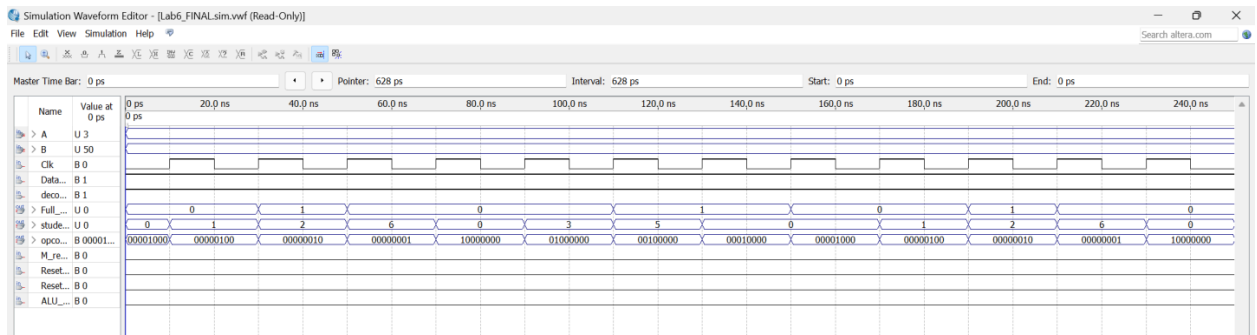


Figure 15: BDF for ALU problem 3

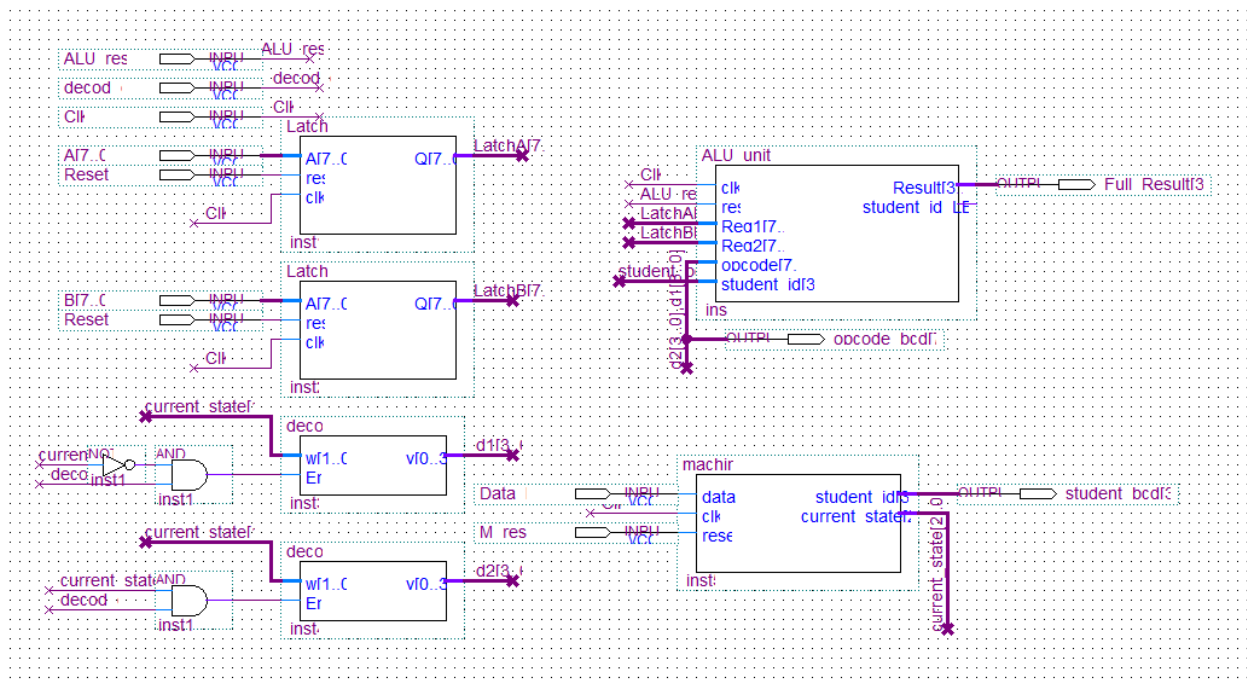


Figure 16: BDF for ALU problem 3

Section 7: Conclusion

The objective of this laboratory was to implement a simplified CPU using an FPGA Altera board; there were three different versions of the ALU to solve three problems with different logical functions. For the first problem, one selects two 8-bit numbers, A and B, based on the Student ID, then uses several logical functions to operate on them. One solves the problem by storing each value in a register. Then, the finite state machine would cycle through its states fed into the decoder, which outputs an Opcode that the ALU could understand. Then, once the ALU completes the operation, the output is sent to the two seven-segment display outputs implemented on the FPGA board.

For problem 2 b), a modification was made to the ALU to perform the logical operation from function table b in 'Figure 3: Function table for ALU problem 2'. Then, the previously stated steps happened with the control unit, which consisted of the FSM and decoder, which selected the states; the ALU, which performed different operations on the same numbers; and the seven-segment displays, which output the visual representation of the output numbers.

For problem 3 b), the ALU was modified to return 'y' if the student ID signal was even and 'n' if otherwise. Overall, this laboratory successfully implemented a simple general processor that performed three sets of logical operations on numbers A and B, verified by the block diagrams and the waveforms for this lab.